Generic vs. non-generic Poseidon2 and Fiat–Shamir

Benchmark report, `prover-ray/crypto/genericbench`

29 October 2026

1 Summary

- The fully generic Poseidon2 code is about **7–10× slower** than the non-generic baseline on KoalaBear and about **3× slower** on MamaBear. KoalaBear's gap is larger because its baseline uses gnark-crypto's AVX-512 assembly.
- The code the prover runs today (`current`: the generic hasher and transcript over gnark-crypto's compression) is **on par with the baseline** (within 3%). Production is not affected.
- The root cause of the generic overhead is Go's implementation of generics: field operations are indirect calls that cannot be inlined, and they force local variables onto the heap. The generic permutation made 222–340 heap allocations per call.
- Branching to concrete KoalaBear code inside the permutation's hot loops removes all allocations from the permutation. With gnark-crypto's formulas replicated, it is about **3.5× faster** than the fully generic code, but still **2.6–2.9× slower** than baseline (2.2–2.6× for hashing and Fiat–Shamir).
- Where gnark-crypto's scalar code uses a cheaper formula, the generic code now uses the same one, so that the comparison measures genericity rather than different formulas. This applies to the internal matrix (Section 6.1). One MamaBear-specific trick is not replicated yet.
- Delegating the whole KoalaBear permutation to gnark-crypto's assembly (`generic-asm`) **matches the baseline** for the permutation itself. Batched `koalabear.Vector` operations inside the hot loops (`generic-vec`) are **about 20× slower** and are a dead end.
- The remaining gap comes from **2 heap allocations per compression** in the generic `Compress` function. Under parallel load (`CompressParallel`, 64 threads) they make the code about 8× slower than baseline, because of **garbage collection**: with the GC switched off the gap shrinks to 1.1×. Marking costs almost nothing. The cost is that each of the GC cycles, which come every few milliseconds, stops the world and makes all 64 goroutines wait in turn: with the GC on, the goroutines spend 57% of their time blocked at the end of the mark phase and run only 15% of the time. Removing these allocations is the next thing to fix.

2 Setup

Machine. Intel Xeon Platinum 8488C, 1 socket, 32 cores / 64 hardware threads, AVX-512 available (so the KoalaBear baseline runs gnark-crypto's assembly path), Linux amd64, Go 1.27.1, gnark-crypto v0.22.0. Nothing else ran on the machine during the benchmarks.

Implementations compared. Each benchmark name ends in `impl=...`:

- `baseline`: non-generic code. For KoalaBear, gnark-crypto's permutation and compression, and the pre-generic MD hasher and transcript (frozen in `legacy_test.go`). For MamaBear, which never had a non-generic hasher, gnark-crypto's permutation under the generic hasher and transcript.
- `current` (KoalaBear only): what the prover runs today.
- `generic`: everything generic, permutation included. It appears under two column names, because the code changed between runs:
  - *fully generic* (Tables 1 and 2): before any branching;
  - *generic + hot-loop branches* (Tables 3 and 4): the permutation's hot loops branch to concrete KoalaBear code (Section 6).
- `generic-vec`, `generic-asm` (KoalaBear only): the two alternative branches of Section 7.

Runs. Six runs, all on the machine above (29 October 2026), each preceded by `TestAgreement`, which checks that all implementations produce bit-identical outputs. It passed every time.

Run	Code	Sizes	Samples
1 (Tables 1, 2)	`3ce6cc6e5`, fully generic	`full`, minus FS rounds=4096/update=16384	6×500 ms
2 (Table 3, "initial")	`6b3d84ad3`, branches and the three paths	`medium`, KoalaBear only	6×200 ms
3 (Tables 3, 4)	`d2b2beda4`: run 2 + gnark-crypto diagonal replicated	`medium`, both fields	6×200 ms
4 (Table 5)	same code as run 3	CompressParallel, KoalaBear, 1–64 threads	6×200 ms
5 (Table 4)	run 3 + `generic-asm-static-call`	`medium`, KoalaBear, Compress/MDHasher/FiatShamir	6×200 ms
6 (Table 5)	same code as run 5	CompressParallel, KoalaBear, 1–64 threads	6×200 ms

The single skipped FiatShamir case took about 40 s per operation in the generic code and dominated the runtime. Tables report the median of 6 samples. Unless noted otherwise, every difference against the baseline is statistically significant according to `benchstat` ($p = 0.002$). The exceptions are `current` on FiatShamir rounds=16/update=8 ($p = 0.70$) and rounds=256/update=1024 ($p = 0.093$) in run 3, both differences of under 1%. The raw output of every run is in `report/data/`.

3 Background: what is benchmarked

All the benchmarks are built on the **Poseidon2 permutation**, the core of the Poseidon2 hash. It takes a fixed-size state of 16 or 24 field elements and scrambles it into another state of the same size. It is reversible, hence "permutation". It runs 27 rounds (6 full, 21 partial), each made of three steps:

1. **Add round keys**: add fixed constants derived from a seed.
2. **S-box**: cube each element, $x \mapsto x^3$. This is the nonlinear step that gives the hash its security. Full rounds cube every element; partial rounds cube only the first.
3. **Linear mixing**: multiply the state by a fixed matrix, so every element affects every other. Full rounds use the "external" matrix built from 4×4 blocks (M4); partial rounds use the cheaper "internal" matrix $\mathbf{1}\mathbf{1}^T + D$.

The other benchmark families are built on the permutation:

- **Compress**: two 8-element blocks form a width-16 state, which is permuted; the second block is then added back (feed-forward). **CompressParallel** runs the same with one goroutine per hardware thread (64 threads on 32 cores).
- **MDHasher**: hashes n elements, 8 at a time, with one compression per chunk.
- **FiatShamir**: a protocol-shaped transcript. Each round absorbs `upd` elements, samples a digest, samples query indices and derives a named challenge.

The permutation dominates every benchmark, so its speed carries through to all of them.

4 Where AVX-512 is used

The KoalaBear baseline is fast because gnark-crypto runs the permutation in AVX-512 assembly. Which of the implementations compared here use that assembly, and where, explains most of the results that follow.

How gnark-crypto chooses. gnark-crypto decides when the permutation is created. It uses the assembly when the width is 16 or 24, the rounds are 6 full and 21 partial, and the CPU supports AVX-512. Every call to `Permutation` then goes straight to the assembly:

Listing 1: gnark-crypto KoalaBear `poseidon2.go` (v0.22.0, excerpt).

```
// in NewParameters
p.hasFast16_6_21 = width == 16 && nbFullRounds == 6 && nbPartialRounds == 21 && (cpu.SupportAVX512 || cpu.
    SupportNEON)
p.hasFast24_6_21 = width == 24 && nbFullRounds == 6 && nbPartialRounds == 21 && cpu.SupportAVX512

func (h *Permutation) Permutation(input []fr.Element) error {
	// ...
	if runtime.GOARCH == "amd64" {
		if h.params.hasFast16_6_21 {
			permutation16_avx512(input, h.params.RoundKeys) // all 27 rounds
			return nil
		}
		if h.params.hasFast24_6_21 {
			permutation24_avx512(input, h.params.RoundKeys)
			return nil
		}
	}
	// ... scalar Go otherwise
```

```
}
```

Every benchmark here uses 6 full and 21 partial rounds, and the benchmark machine has AVX-512, so on KoalaBear both the width-16 and the width-24 permutation take this path. The compression function `vortex.CompressPoseidon2` calls the same width-16 permutation.

Per implementation.

Implementation	AVX-512 in each permutation?	How
KoalaBear `baseline`	yes	gnark-crypto's assembly, all 27 rounds in one call
`current`	yes	the same: generic hasher and transcript over gnark-crypto's compression
`generic-asm`	yes, the same assembly	`Permute` hands the whole state to gnark-crypto (Listing 6)
`generic-vec`	partly	round keys, full S-box layer and internal matrix as separate `koalabear.Vector` calls on 16 elements each (Listing 5)
`generic` (fully generic or branched)	no	scalar Go; the Go compiler does not emit AVX-512 itself
MamaBear `baseline` and `generic`	no	gnark-crypto has no MamaBear assembly

For `generic-asm`, only the code around the permutation is generic Go: copying the blocks in, the feed-forward additions, and (Section 8) the two heap allocations in `Compress`. The CPU profile at 64 threads with the GC off confirms it: 68% of the samples are in `permutation16_avx512`.

One state at a time. The assembly holds one width-16 state in a single 512-bit register (16 lanes of 32 bits) and runs all rounds on it in place. gnark-crypto also has versions that permute 16 independent states at once (`Permutation16x24`, `permutation16x16xN_avx512`), but `Compress` and everything benchmarked here use the one-state version. **CompressParallel adds no SIMD across calls**: it runs the same one-state permutation on many goroutines, so its only parallelism is threads. Single-threaded, `baseline` and `generic-asm` therefore cost about the same per compression (585 and 650 ns), and the gap under parallel load comes entirely from the garbage collector (Section 8).

Consequences for the comparison. KoalaBear's `generic`/`baseline` ratio compares scalar Go with assembly, so it mixes the cost of genericity with the lack of SIMD. MamaBear has no assembly on either side, which makes it the better measure of genericity alone (Section 5). On a CPU without AVX-512, the KoalaBear baseline would also fall back to scalar Go, and every KoalaBear ratio here would be smaller.

5 Results 1: fully generic code

Table 1: KoalaBear, fully generic code (before any branching). Ratios are time relative to `baseline`.

Benchmark	baseline	fully generic	ratio	`current`	ratio	allocs `baseline`	allocs fully generic	allocs `current`
Permutation (width=16)	569 ns	5.02 µs	8.82×	–	–	0	222	–
Permutation (width=24)	705 ns	7.25 µs	10.3×	–	–	0	298	–
Compress	585 ns	5.09 µs	8.71×	–	–	0	224	–
CompressParallel	12.6 ns	1.32 µs	105×	–	–	0	224	–
MDHasher (n=8)	851 ns	5.03 µs	5.91×	668 ns	0.79×	5	225	1
MDHasher (n=64)	5.04 µs	40.1 µs	7.94×	4.9 µs	0.97×	5	1 793	1
MDHasher (n=4096)	308 µs	2.54 ms	8.23×	311 µs	1.01×	5	114 689	1
MDHasher (n=65536)	4.92 ms	40.7 ms	8.27×	4.97 ms	1.01×	5	1 835 009	1
MDHasher (n=1048576)	78.6 ms	615 ms	7.82×	79.4 ms	1.01×	5	29 360 129	1
FiatShamir (rounds=16, upd=8)	390 µs	2.86 ms	7.35×	399 µs	1.02×	254	126 173	731
FiatShamir (rounds=16, upd=1024)	1.62 ms	13 ms	8.06×	1.64 ms	1.01×	254	581 351	731
FiatShamir (rounds=16, upd=16384)	20.1 ms	167 ms	8.30×	20.3 ms	1.01×	257	7 462 752	734
FiatShamir (rounds=256, upd=8)	6.23 ms	45.6 ms	7.32×	6.37 ms	1.02×	3 851	2 018 614	11 528
FiatShamir (rounds=256, upd=1024)	25.9 ms	208 ms	8.01×	26.2 ms	1.01×	3 856	9 301 452	11 532
FiatShamir (rounds=256, upd=16384)	321 ms	2.66 s	8.28×	324 ms	1.01×	3 906	119 403 792	11 582
FiatShamir (rounds=4096, upd=8)	98.7 ms	702 ms	7.11×	101 ms	1.03×	65 524	32 301 470	188 410
FiatShamir (rounds=4096, upd=1024)	412 ms	3.19 s	7.73×	417 ms	1.01×	65 574	148 825 871	188 444

How to read the tables. Each ratio is time relative to `baseline`: 1× is the same speed, 9× means nine times as long, and below 1× is faster. The allocs columns give heap allocations per operation.

Findings.

Table 2: MamaBear, fully generic code. The MamaBear baseline is gnark-crypto's (non-assembly) permutation under the generic hasher.

Benchmark	baseline	fully generic	ratio	allocs `baseline`	allocs fully generic
Permutation (width=16)	2.08 µs	6.9 µs	3.31×	0	250
Permutation (width=24)	3.39 µs	9.82 µs	2.90×	0	340
Compress	2.2 µs	7.05 µs	3.20×	2	252
CompressParallel	289 ns	1.98 µs	6.87×	2	252
MDHasher (n=8)	2.14 µs	6.82 µs	3.18×	3	253
MDHasher (n=64)	17.2 µs	54.7 µs	3.17×	17	2 017
MDHasher (n=4096)	1.13 ms	3.51 ms	3.09×	1 025	129 025
MDHasher (n=65536)	18.1 ms	54.4 ms	3.00×	16 385	2 064 385
MDHasher (n=1048576)	286 ms	791 ms	2.77×	262 145	33 030 145
FiatShamir (rounds=16, upd=8)	1.3 ms	3.86 ms	2.98×	1 788	141 792
FiatShamir (rounds=16, upd=1024)	5.83 ms	17.9 ms	3.07×	5 853	653 870
FiatShamir (rounds=16, upd=16384)	74.1 ms	227 ms	3.06×	67 312	8 395 418
FiatShamir (rounds=256, upd=8)	20.8 ms	61.4 ms	2.96×	28 444	2 268 504
FiatShamir (rounds=256, upd=1024)	92.9 ms	282 ms	3.03×	93 491	10 461 752
FiatShamir (rounds=256, upd=16384)	1.19 s	3.59 s	3.03×	1 076 815	134 326 653
FiatShamir (rounds=4096, upd=8)	330 ms	935 ms	2.83×	458 600	36 299 124
FiatShamir (rounds=4096, upd=1024)	1.47 s	4.27 s	2.90×	1 499 170	167 389 495

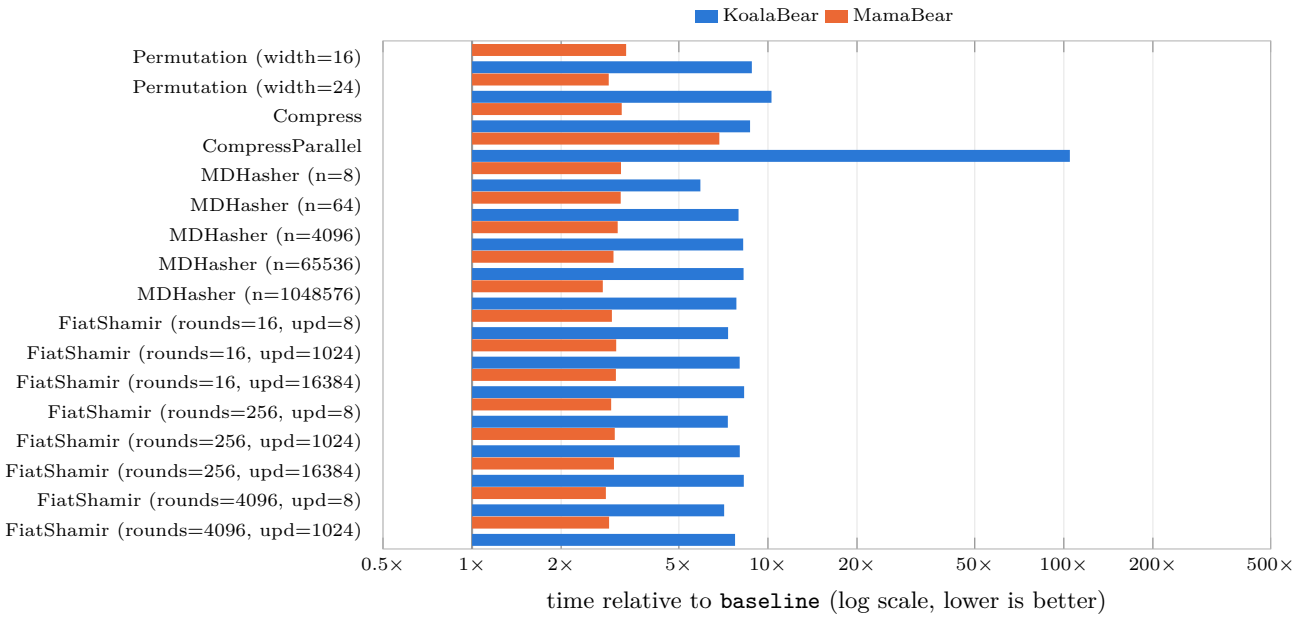


Figure 1: Fully generic code: time relative to each fields `baseline` (Tables 1 and 2). The vertical line marks parity.

- KoalaBear `generic` is 5.9–10.3× slower than baseline, and MamaBear `generic` 2.8–3.3×. Without assembly on the MamaBear side, the MamaBear figure is the better measure of the cost of genericity itself.
- `current` is on par with baseline: from 21% faster (MDHasher, $n = 8$) to about 3% slower (FiatShamir). It also makes 80% fewer allocations in MDHasher, but about 2.9× more in FiatShamir. The absolute numbers are small (731 vs. 254 at 16 rounds), but this may deserve a look.
- The generic code allocates heavily: 222–340 allocations per permutation, about 28 per absorbed element in the KoalaBear MDHasher, and up to 167 million per FiatShamir operation.

Why the generic code is slow. Go does not compile a generic function separately for every type. It compiles it once per "GC shape" (here, every `[1]uint32`-shaped field shares the code) and passes a dictionary of the type's methods. So inside `sBox`, `matMulM4` and the other helpers, each `PT(x).Mul`, `PT(x).Add`, ... is an *indirect call* through that dictionary. The compiler cannot inline it, and it cannot see what the callee does with the pointers it receives, so it moves the operands to the heap. That causes both the per-operation call overhead and the hundreds of allocations per permutation.

Example: the internal matrix. Every version below implements the same step of a partial round, multiplying the state by $\mathbf{1}\mathbf{1}^T + D$: add up all elements, then set each element to the sum plus itself times its diagonal entry. This is the fully generic version. Each `PT(...)` call goes through the dictionary, and `sum` and `tmp` are moved to the heap:

Listing 2: Fully generic `matMulInternal` (commit `3ce6cc6e5`).

```
func (h *Permutation[T, PT]) matMulInternal(input []T) {
	var sum T
	for i := range input {
		PT(&sum).Add(&sum, &input[i]) // indirect call
	}
	var tmp T
	for i := range input {
		PT(&tmp).Mul(&input[i], &h.params.InternalDiag[i]) // indirect call
		PT(&input[i]).Add(&sum, &tmp) // indirect call
	}
}
```

For comparison, gnark-crypto's own scalar fallback (used when there is no AVX-512) is written for one field and one width. It never multiplies by the diagonal: the diagonal entries are small values such as ± 2, $\pm\frac{1}{2}$ or 2^{-8}, so it uses doublings, halvings and shifts instead. Since run 3, the generic code and the KoalaBear branch use the same formula (Section 6.1); Tables 1 and 2 predate that change.

Listing 3: gnark-crypto KoalaBear `matMulInternalInPlace`, width 16 (excerpt).

```
var sum fr.Element
sum.Set(&input[0])
for i := 1; i < h.params.Width; i++ {
	sum.Add(&sum, &input[i])
}
// mul by diag16: [-2, 1, 2, 1/2, 3, 4, -1/2, -3, -4, 1/2^8, ...]
var temp fr.Element
input[0].Sub(&sum, temp.Double(&input[0]))
input[1].Add(&sum, &input[1])
input[2].Add(&sum, temp.Double(&input[2]))
temp.Set(&input[3]).Halve()
input[3].Add(&sum, &temp)
// ...
input[9].Add(&sum, temp.Mul2ExpNegN(&input[9], 8))
// ...
```

6 Results 2: branching to concrete KoalaBear code in the hot loops

Each hot helper of the permutation (`sBox`, `addRoundKey`, `matMulExternal` with M4, `matMulInternal`) now starts with `if h.kb { ...; return }`. The flag `kb` is computed once in `NewPermutation` by a type check. The KoalaBear branch reinterprets its operands as `[]koalabear.Element` (via `unsafe`, without copying) and runs concrete loops that the compiler can inline and keep on the stack. The check runs once per helper call; putting it inside the per-element loops would only add work. Other fields keep the generic code.

Listing 4: `matMulInternal` with the hot-loop branch (`crypto/poseidon2`). The vector case belongs to Section 7.

```
// permutation.go: the generic helper branches once, on entry.
func (h *Permutation[T, PT]) matMulInternal(input []T) {
	if h.kb { // set once in NewPermutation: is T koalabear.Element?
		if h.kbPath == KoalaBearVector {
			kbMatMulInternalVec(asKB(input), asKB(h.params.InternalDiag))
			return
		}
		kbMatMulInternal(asKB(input))
		return
	}
	// ... generic code for every other field ...
}

// koalabear.go: reinterpret []T as []koalabear.Element, without copying.
func asKB[T any](s []T) []koalabear.Element {
	return unsafe.Slice((*koalabear.Element)(unsafe.Pointer(unsafe.SliceData(s))), len(s))
}

// koalabear.go: gnark-crypto's code, copied; direct calls the compiler inlines.
func kbMatMulInternal(input []koalabear.Element) {
	var sum koalabear.Element // stays on the stack
	sum.Set(&input[0])
	for i := 1; i < len(input); i++ {
		sum.Add(&sum, &input[i])
	}
	var temp koalabear.Element
	input[0].Sub(&sum, temp.Double(&input[0]))
	input[1].Add(&sum, &input[1])
	// ... as in Listing 3, for width 16 and 24 ...
}
```

Table 3: KoalaBear, the hot-loop branches before (run 2) and after (run 3) replicating gnark-crypto's internal-matrix diagonal. Each ratio is relative to the `baseline` of the same run; the baseline column shows run 3.

Benchmark	baseline	branches (initial)	ratio	branches, gnark diagonal	ratio	allocs, gnark diagonal
Permutation (width=16)	570 ns	1.93 µs	3.39×	1.46 µs	2.57×	0
Permutation (width=24)	705 ns	2.64 µs	3.75×	2.08 µs	2.94×	0
Compress	586 ns	2.02 µs	3.45×	1.54 µs	2.64×	2
CompressParallel	12.8 ns	271 ns	20.8×	238 ns	18.7×	2
MDHasher (n=64)	4.97 µs	16.4 µs	3.28×	11.8 µs	2.37×	17
MDHasher (n=4096)	308 µs	1.04 ms	3.38×	790 µs	2.57×	1 025
MDHasher (n=65536)	4.91 ms	16.7 ms	3.37×	12.6 ms	2.57×	16 385
FiatShamir (rounds=16, upd=8)	134 µs	384 µs	2.87×	298 µs	2.22×	698
FiatShamir (rounds=16, upd=1024)	1.36 ms	4.54 ms	3.32×	3.48 ms	2.55×	4 763
FiatShamir (rounds=256, upd=8)	2.13 ms	6.12 ms	2.86×	4.82 ms	2.26×	11 014
FiatShamir (rounds=256, upd=1024)	21.8 ms	72.4 ms	3.33×	55.6 ms	2.55×	76 055

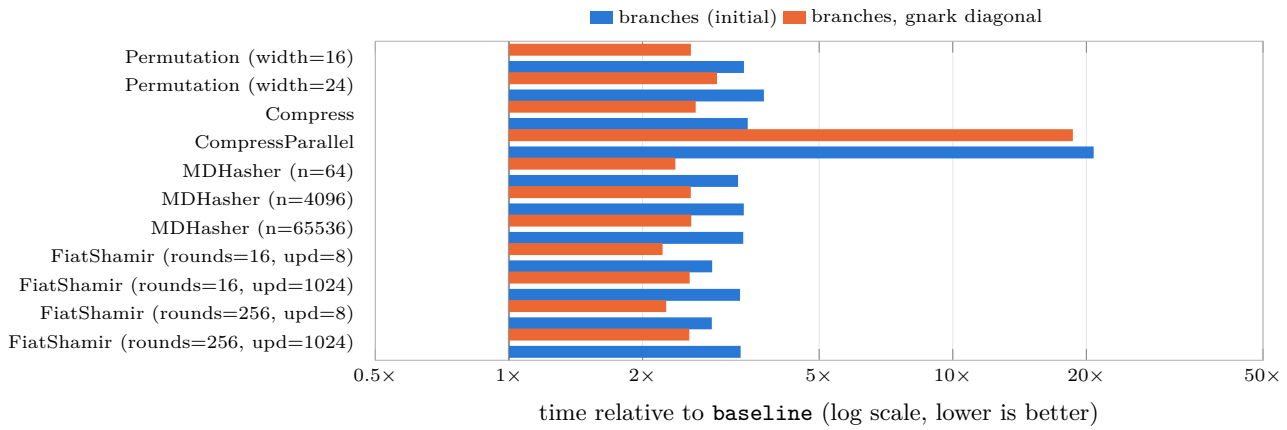


Figure 2: KoalaBear hot-loop branches, time relative to `baseline`, before and after replicating gnark-crypto's internal-matrix diagonal (Table 3).

Findings. The permutation is now allocation-free. Its first version (run 2) was 3.4–3.8× slower than baseline. With gnark-crypto's internal-matrix formula replicated (run 3, Section 6.1), it is 2.6–2.9× slower, and the hashing and Fiat–Shamir benchmarks 2.2–2.6×. The remaining gap is the AVX-512 assembly: one AVX-512 instruction processes 16 elements at once, which is exactly one width-16 state, and gnark-crypto's assembly keeps the whole state in registers for all 27 rounds. Scalar Go code cannot do that. The remaining allocations (2 per compression) are analysed in Section 8.

6.1 Replicating gnark-crypto's optimisations

A comparison with gnark-crypto should measure the cost of genericity, not of different formulas. Each helper was therefore checked against gnark-crypto's scalar code for both fields:

Step	gnark-crypto's scalar code	Our code
S-box, round keys, M4, external matrix	same formulas	identical
Internal matrix: diagonal	doublings, `Halve`, `Mul2ExpNegN` instead of `Mul`	replicated in run 3 (generic and KoalaBear branch)
Internal matrix: sum, MamaBear only	raw 64-bit sum, a single reduction at the end (`sumState`)	**not replicated**

To replicate the diagonal in the generic code, `generic.Element` gained two methods that both fields already have, `Halve` and `Mul2ExpNegN`. The KoalaBear branch is a line-by-line copy of gnark-crypto's function. The generic version applies the 2^{-n} entries in a short loop over a table of shifts rather than unrolling them.

The effect differs by path. The KoalaBear branch became about 24% faster (Table 3). The fully generic MamaBear permutation did not: 7.41 µs in run 3, compared with 6.90 µs in run 1, a difference within the noise of those runs. In the generic code every field operation is an indirect call, and the formula replaces one multiplication by two to four cheaper operations, so the number of calls, which dominates, goes up. MamaBear's `sumState` relies on the field's internal representation and cannot be expressed through `generic.Element`; replicating it needs a MamaBear branch or a new interface method.

7 Results 3: vector operations vs. assembly

Two ways to close the assembly gap were implemented side by side. The choice is made with `SetKoalaBearPath`:

- `KoalaBearScalar` (default, column "generic + hot-loop branches"): the concrete loops of Section 6.
- `KoalaBearVector` (`generic-vec`): the same branches, but the round keys, the full S-box layer and the internal matrix use gnark-crypto's batched `koalabear.Vector` operations (`Add`, `Mul`, `Sum`), which run AVX-512 loops over slices. M4 and the external matrix stay scalar because they mix neighbouring elements. For width 24, the last 8 elements fall back to scalar code because the vector operations work in blocks of 16.
- `KoalaBearAsm` (`generic-asm`): the branch sits at the top of `Permute` and hands the whole permutation to gnark-crypto's AVX-512 function.

Listing 5: The vector path: `matMulInternal` as three batched calls (`koalabear.go`).

```
func kbMatMulInternalVec(input, diag []koalabear.Element) {
	v := koalabear.Vector(input)
	sum := v.Sum() // AVX-512 call 1
	var buf [maxWidth]koalabear.Element // stack scratch
	sums := koalabear.Vector(buf[:len(input)])
	for i := range sums {
		sums[i] = sum
	}
	v.Mul(v, diag) // AVX-512 call 2
	v.Add(v, sums) // AVX-512 call 3
}
```

Listing 6: The assembly path: the branch sits at the top of `Permute`, so no helper runs at all (`permutation.go`).

```
func (h *Permutation[T, PT]) Permute(input []T) error {
	// ...
	if h.kbAsm != nil { // set by SetKoalaBearPath(KoalaBearAsm)
		return h.kbAsm.Permutation(asKB(input)) // gnark-crypto: all 27 rounds in AVX-512
	}
	h.matMulExternal(input)
	// ... rounds, each calling the helpers of Listing 4 ...
}
```

Table 4: KoalaBear, all code paths side by side (run 3, `medium` scale). The hot-loop branches include the gnark-crypto internal-matrix diagonal. `generic-asm-static-call` is from run 5 and its ratio is relative to run 5's `baseline`; it changes only the compression, so it has no Permutation rows.

Benchmark	baseline	generic + hot-loop branches	ratio	generic-vec	ratio	generic-asm	ratio	current	ratio	generic-asm-static-call	ratio	allocs generic-asm	allocs generic-asm-static-call
Permutation (width=16)	570 ns	1.46 µs	2.57×	11.6 µs	20.4×	574 ns	1.01×	–	–	–	–	0	–
Permutation (width=24)	705 ns	2.08 µs	2.94×	12.4 µs	17.6×	704 ns	1.00×	–	–	–	–	0	–
Compress	586 ns	1.54 µs	2.64×	11.7 µs	19.9×	653 ns	1.11×	–	–	594 ns	1.01×	2	0
CompressParallel	12.8 ns	238 ns	18.7×	584 ns	45.7×	106 ns	8.27×	–	–	13.1 ns	1.05×	2	0
MDHasher (n=64)	4.97 µs	11.8 µs	2.37×	93.1 µs	18.7×	5.31 µs	1.07×	4.87 µs	0.98×	4.92 µs	0.98×	17	1
MDHasher (n=4096)	308 µs	790 µs	2.57×	5.95 ms	19.3×	338 µs	1.10×	311 µs	1.01×	313 µs	1.02×	1 025	1
MDHasher (n=65536)	4.91 ms	12.6 ms	2.57×	95.1 ms	19.4×	5.36 ms	1.09×	4.97 ms	1.01×	4.99 ms	1.02×	16 385	1
FiatShamir (rounds=16, upd=8)	134 µs	298 µs	2.22×	2.09 ms	15.6×	144 µs	1.07×	134 µs	1.00×	136 µs	1.01×	698	346
FiatShamir (rounds=16, upd=1024)	1.36 ms	3.48 ms	2.55×	25.8 ms	19.0×	1.5 ms	1.10×	1.37 ms	1.01×	1.38 ms	1.01×	4 763	346
FiatShamir (rounds=256, upd=8)	2.13 ms	4.82 ms	2.26×	33.5 ms	15.7×	2.28 ms	1.07×	2.15 ms	1.01×	2.18 ms	1.01×	11 014	5 382
FiatShamir (rounds=256, upd=1024)	21.8 ms	55.6 ms	2.55×	414 ms	19.0×	23.9 ms	1.10×	21.9 ms	1.00×	22.1 ms	1.02×	76 055	5 387

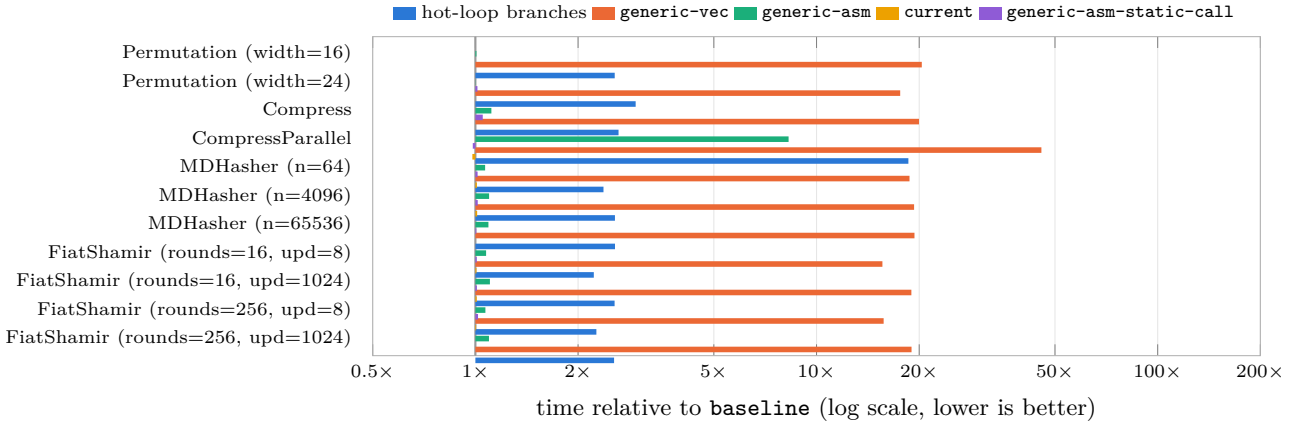


Figure 3: KoalaBear code paths, time relative to `baseline` (Table 4). `current` exists only for MDHasher and FiatShamir; `generic-asm-static-call` (run 5, against run 5's `baseline`) only changes the compression, so it has no Permutation bars.

Findings.

- `generic-asm` matches the baseline permutation (1.00–1.01×). The Compress, MDHasher and FiatShamir benchmarks remain 7–11% slower than baseline. That overhead comes from the generic code around the permutation.
- `generic-vec` is about 20× slower than baseline, and about 8× slower than the scalar branch it was meant to improve. (It still multiplies by the diagonal; it was not updated in run 3.) A CPU profile shows almost all the time inside the vector routines (75% in the 21 partial rounds), at about 150 ns per call on 16 elements. The vector routines are designed for long arrays; on a 16-element state the cost of each call outweighs the work. The likely cause is the penalty the CPU pays when switching between AVX-512 and ordinary instructions, but this has not been confirmed.

Why the two cannot be combined in one permutation. Both paths live in the same codebase and are compiled into the same binary; nothing conflicts. A given permutation can use only one of them, though: the assembly function performs all 27 rounds in a single call, so once `Permute` hands it the state, none of the per-step helpers run and there is nowhere for a vector branch to apply. gnark-crypto does not expose the individual rounds of its assembly. This is the underlying trade-off: branching as low as possible and matching the baseline's speed pull in opposite directions, because the fast code exists only as one whole-permutation block.

8 Why CompressParallel is so slow

Even with the assembly permutation, `CompressParallel` is about 8× slower than baseline at 64 threads (Table 4). To find out why, it was re-run at 1 to 64 threads, with the Go garbage collector (GC) on and off, and profiled at 64 threads. The machine has 1 socket of 32 cores, each core running 2 hardware threads, so 64 threads means 32 physical cores. The data is in `report/data/`.

8.1 Where the allocations come from

A memory profile (`report/data/heap/mem-profile.txt`) attributes 99.94% of the allocated bytes to two lines of the generic `Compress` (`crypto/poseidon2/hasher.go`):

Listing 7: Generic `Compress`. The two marked arrays are moved to the heap.

```
func Compress[T any, PT generic.Element[T]](perm *Permutation[T, PT], state, block Octuplet[T]) Octuplet[T]
    {
      var x [CompressionWidth]T // ALLOCATES: 16 elements, 64 bytes
      copy(x[:BlockSize], state[:])
      copy(x[BlockSize:], block[:])
      if err := perm.Permute(x[:]); err != nil { // x escapes here
            panic(err)
      }
      res := block // ALLOCATES: 8 elements, 32 bytes
      for i := range res {
            PT(&res[i]).Add(&res[i], &x[BlockSize+i]) // res escapes here
      }
      return res
}
```

Listing 8: gnark-crypto `vortex.CompressPoseidon2`: the same steps, nothing allocated.

```
func CompressPoseidon2(a, b Hash) Hash {
      res := Hash{}
      var x [16]koalabear.Element
      copy(x[:], a[:])
      copy(x[8:], b[:])
      copy(res[:], x[8:])
      if err := compressPerm.Permutation(x[:]); err != nil {
            panic(err)
      }
      for i := range res {
            res[i].Add(&res[i], &x[8+i])
      }
      return res
}
```

That is 2 allocations and 96 bytes per compression, for every generic path. These are fixed-size local arrays and should stay on the stack, as they do in gnark-crypto's almost identical `CompressPoseidon2`. They escape for the reason given in Section 5: `x` is passed to `perm.Permute` and `res` to `PT(&res[i]).Add`, both indirect calls the compiler cannot see through, so it assumes the pointers may be kept and moves both arrays to the heap.

8.2 Scaling with the number of threads

Table 5: KoalaBear CompressParallel, nanoseconds per compression (lower is better) by number of threads (`-cpu`), with the garbage collector on (default) and off (`GOGC=off`). Ratios are relative to `baseline` at the same thread count. `baseline` allocates nothing, so the GC setting does not affect it. `generic-asm-static-call` comes from run 6 and its ratios are relative to run 6's `baseline`, measured with the same GC setting.

		`generic-asm`				generic + hot-loop branches				`generic-asm-static-call` (run 6)			
Threads	baseline	GC on	ratio	GC off	ratio	GC on	ratio	GC off	ratio	GC on	ratio	GC off	ratio
1	584	652	1.1×	647	1.1×	1560	2.7×	1620	2.8×	595	1.0×	601	1.0×
2	292	335	1.1×	325	1.1×	785	2.7×	828	2.8×	298	1.0×	299	1.0×
4	146	173	1.2×	158	1.1×	395	2.7×	397	2.7×	149	1.0×	149	1.0×
8	73.3	106	1.5×	80.8	1.1×	212	2.9×	193	2.6×	74.7	1.0×	74.6	1.0×
16	38.8	82.3	2.1×	42.1	1.1×	144	3.7×	104	2.7×	37.9	1.0×	37.9	1.0×
32	21.7	80.4	3.7×	24.0	1.1×	142	6.5×	58.2	2.7×	21.6	1.0×	21.7	1.0×
48	16.0	90.0	5.6×	18.1	1.1×	170	11×	54.3	3.4×	17.1	1.1×	15.9	1.0×
64	12.8	103	8.1×	14.6	1.1×	214	17×	51.2	4.0×	14.2	1.1×	12.7	1.0×

Table 5 shows the key result:

- **With the GC on**, `generic-asm` keeps up with baseline only up to about 4 threads. It reaches its best time at 32 threads (80 ns) and then gets *slower* as threads are added (90 ns at 48 threads, 103 ns at 64). Baseline keeps improving to 12.8 ns, so the ratio grows from 1.1× to 8.1×. The branched scalar path behaves the same way (best 142 ns at 32 threads, 17× at 64).
- **With the GC off**, `generic-asm` scales as well as baseline: 1.1× at every thread count, 14.6 ns at 64 threads. The scalar path stays at 2.6–4.0×, close to its single-threaded ratio.

So the parallel slowdown is caused almost entirely by **garbage collection**, not by allocation itself and not by the permutation. Allocating without collecting costs about 10–15% (the remaining gap with the GC off).

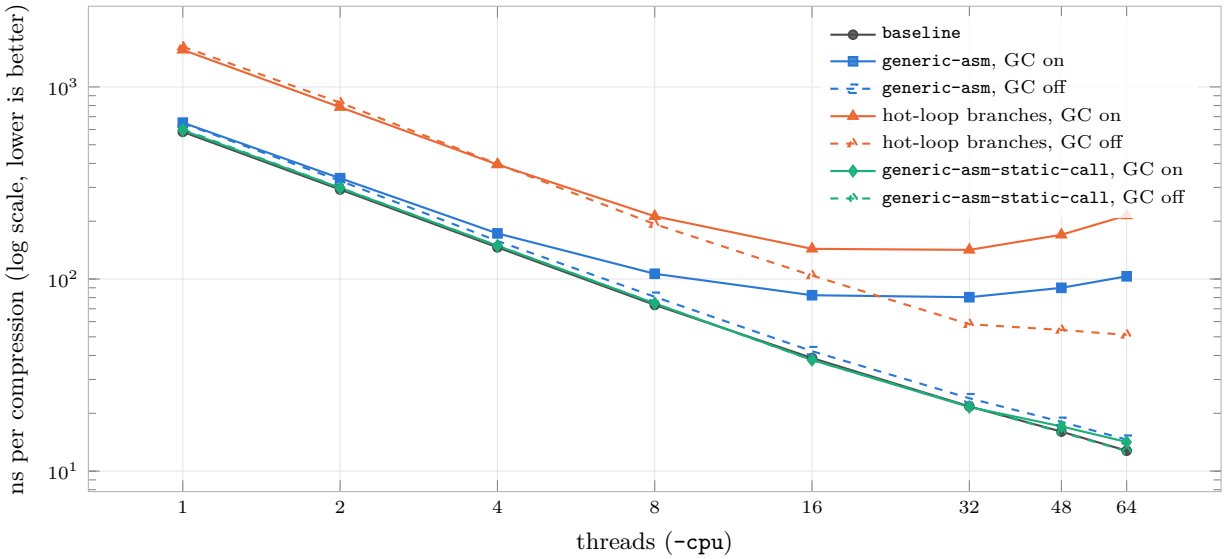


Figure 4: KoalaBear CompressParallel scaling (Table 5). Solid: garbage collector on (default); dashed: off. Colour follows the implementation. Green: `generic-asm-static-call` (Section 8.4, run 6), which lies on the baseline with the GC on or off.

8.3 Heap and GC analysis

The measurements in this section are all for `generic-asm` on the same code as run 3. Data, tools and commands are in `report/data/heap/`. Five sources were used:

- a memory profile of the benchmark (`mem-profile.txt`);
- a small harness (`gcharness/`) that runs compressions on N goroutines and reads the Go runtime's own GC accounting (`runtime/metrics`) before and after (`gc-scaling.tsv`, `gc-gogc.tsv`);
- an execution trace of the benchmark at 64 threads, analysed with `stw.py` and `states.py` (`trace-analysis.txt`);
- the kernel's per-thread scheduler counters (`schedstat.txt`);
- the Go 1.27.1 runtime source, to connect what the trace shows to the code.

The heap. The memory profile attributes 99.94% of allocated bytes to `Compress`: 989 MB to `var x` (64-byte objects) and 498 MB to `res := block` (32-byte objects) in a 2 s run. The in-use heap is under 1 MB, since every object is garbage as soon as `Compress` returns. The live heap seen by the GC is 1–8 MB, so the default `GOGC=100` sets a heap goal of only 4–15 MB, and a cycle starts every few milliseconds (Table 6).

Table 6: `generic-asm` compressions on N goroutines, GC on, as reported by the Go runtime (`runtime/metrics`). Stop-the-world (STW) time is the share of wall-clock time with the world stopped; each GC cycle stops it twice. Mark CPU is assists plus background mark workers, as a share of CPU time.

Threads	ns/op	ms between cycles	STW, % of wall	mean STW (µs)	mark CPU
1	635	15.2	0.1%	5	0.9%
2	326	8.25	0.5%	22	1.0%
4	164	4.83	1.2%	29	1.0%
8	89.8	2.46	3.8%	47	1.2%
16	99.4	2.68	24.3%	326	0.9%
32	103	3.59	28.1%	505	0.7%
48	99.6	5.64	46.8%	1319	0.6%
64	106	7.75	56.1%	2177	0.5%

Marking costs almost nothing; stopping the world does. Mark work (assists and background workers) stays at about 1% of CPU at every thread count: with a heap this small, there is nothing to mark. What grows is the time with the world stopped. Up to 8 threads it is under 4% of wall-clock time and each pause lasts about 50 µs. Between 8 and 16 threads it jumps to 24%, and at 64 threads the world is stopped **56% of the time**, about 2.2 ms per pause. This jump is where `generic-asm` stops improving with more threads (90 ns at 8 threads, 99–106 ns from 16 to 64).

The threads are asleep, not busy. The kernel's counters show that with the GC on, the process ran on about 8 of the 64 cores on average, against 57 with the GC off. The threads sleep on their own (about 53 000 voluntary

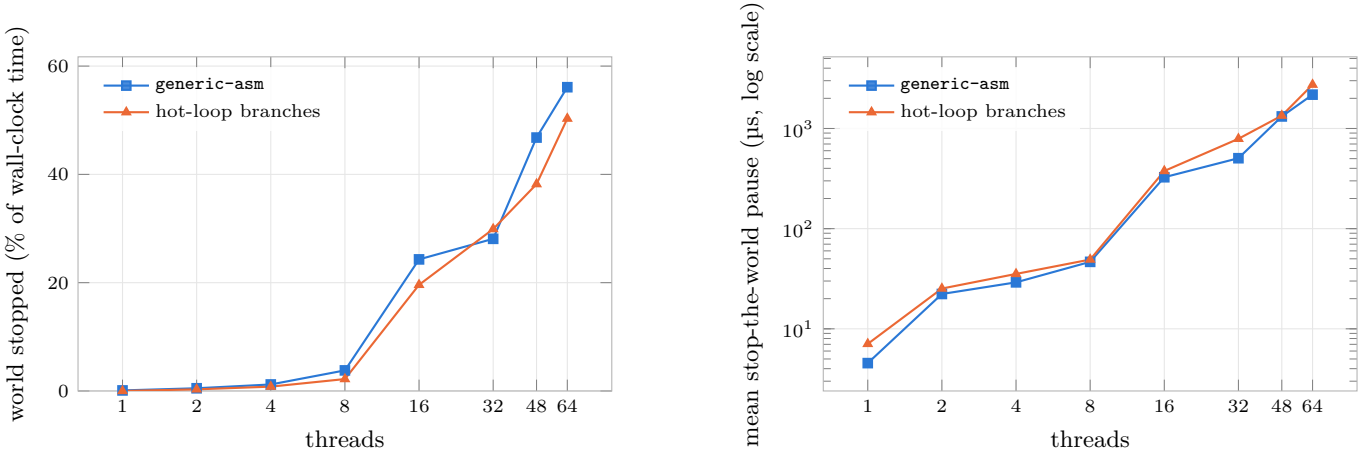


Figure 5: Stop-the-world time with the GC on, by number of threads (Table 6). Both generic paths jump between 8 and 16 threads.

context switches per second); they are not waiting for a CPU (20 ms of run-queue wait in 2 s). So the time is lost to goroutines blocked inside the Go runtime. (The runtime's own CPU estimate counts stopped time as busy on every thread, so its "GC pause CPU" figure overstates the CPU used.)

Where the goroutines block. The execution trace gives the state of each benchmark goroutine over time:

Share of goroutine time	State	Where
57%	blocked on a semaphore	`gcMarkDone`, called from `gcAssistAlloc`
18%	runnable, preempted	waiting to get a thread back
15%	running	
7%	blocked on a semaphore	`gcStart`
3%	runnable, just woken	
1%	waiting for mark work	`gcParkAssist`

Every one of these waits begins at one of the two allocations in `Compress`. Read against the runtime source, each GC cycle goes like this:

1. **Start.** The allocation that crosses the heap goal calls `gcStart`. With 64 goroutines allocating, many cross it at the same moment. Only one may start the cycle (`work.startSema`), and the others block (7%). Starting the cycle stops the world (sweep termination).
2. **Mark.** Goroutines that allocate during marking must do some marking work first (`gcAssistAlloc`). With almost nothing to mark, the first assists already find the work exhausted. Each of them then calls `gcMarkDone` to end the phase.
3. **End of marking.** `gcMarkDone` is serialised by `work.markDoneSema`. The goroutine holding it flushes every P's buffers (`forEachP`, which waits for all 64 Ps to reach a safe point) and then stops the world again for mark termination. Meanwhile every other allocating goroutine has reached the same point and waits on the semaphore (57%).
4. **Restart.** When the world restarts, the waiting goroutines are woken together and queue for the 64 Ps (the 18% runnable time), and a few milliseconds later the next cycle begins.

So with the GC on, each cycle effectively serialises all 64 goroutines twice, and the cycles come every few milliseconds. More threads make it worse, because they reach the heap goal sooner and there are more of them to stop and to queue.

The trace also shows that the goroutines themselves stop quickly once a stop is requested (median 49 µs, 90th percentile 374 µs), and that the median stop-the-world window lasts 1.5 ms. The trace does not say what the rest of each window is spent on; in the sample examined, about 0.8 ms of a 1.4 ms window passes with no events at all. This part is not explained.

It is the frequency of cycles. Table 7 raises `GOGC`, which lets the heap grow further before each cycle. The cost falls steadily with the number of cycles: from 8.3× baseline at `GOGC=100` to 1.9× at 1600 and 1.4× at 6400. With the GC off, only the cost of allocating remains (16 ns, 1.3×). Running the benchmark itself with the same settings gives the same picture (119, 51, 24 and 15 ns for `GOGC` 100, 400, 1600 and off; baseline 12.6 ns throughout).

Table 7: `generic-asm` compressions on 64 goroutines for several values of `GOGC`. The ratio is against `baseline` at 64 threads (12.8 ns, Table 5).

GOGC	ns/op	ratio	ms between cycles	heap goal (MB)	STW, % of wall
100	107	8.3×	8.34	15.5	54.4%
200	74.6	5.8×	8.81	16.7	54.2%
400	49.6	3.9×	11.4	33.3	45.6%
800	34.4	2.7×	15.8	45.1	37.0%
1600	24.6	1.9×	23.6	126	25.7%
6400	17.9	1.4×	65	268	8.0%
off	16.1	1.3×	–	–	0.0%

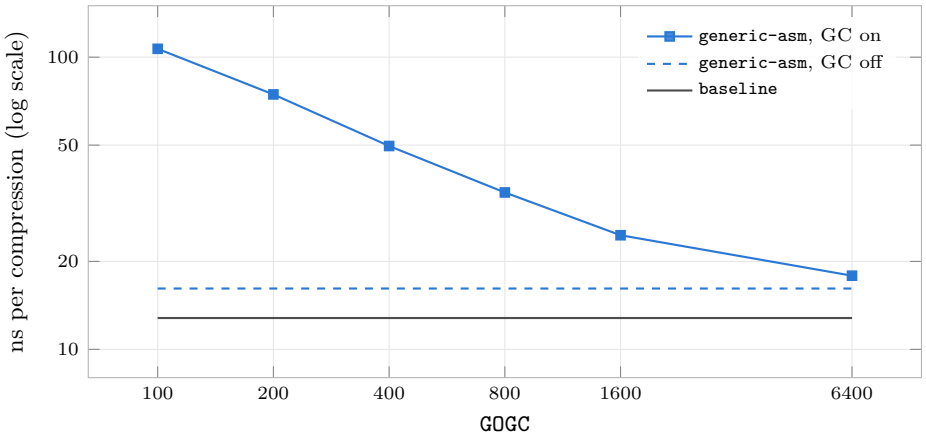


Figure 6: `generic-asm` at 64 threads as `GOGC` is raised (Table 7). Fewer GC cycles, lower cost.

8.4 Experiment: an allocation-free `Compress`

To confirm that the two allocations are the whole story, `Compress` was given a KoalaBear fast path for `generic-asm`. When the permutation delegates to gnark-crypto's assembly, the state and the result are held as concrete `[16]koalabear.Element` and `[8]koalabear.Element` arrays, and the feed-forward calls `koalabear.Element.Add` directly. With no indirect calls, escape analysis keeps both arrays on the stack.

Listing 9: The fast path added at the top of `Compress` (`report/data/heap/experiment/`).

```
if perm.kbAsm != nil {
	var y [CompressionWidth]koalabear.Element
	copy(y[:BlockSize], asKB(state[:]))
	copy(y[BlockSize:], asKB(block[:]))
	if err := perm.kbAsm.Permutation(y[:]); err != nil {
		panic(err)
	}
	res := block
	r := asKB(res[:])
	for i := range r {
		r[i].Add(&r[i], &y[BlockSize+i])
	}
	return res
}
```

`TestAgreement` passes with the change. `CompressParallel` then makes 0 allocations and runs at 12.9 ns per compression at 64 threads with the GC on, against 12.8 ns for baseline and 103 ns without the fix. It tracks baseline at every thread count (green line in Figure 4). The patch, the patched file and the benchmark output of this first experiment are in `report/data/heap/experiment/`.

`generic-asm-static-call`. The fix is now a separate implementation, so that `generic-asm` keeps its measurements. `poseidon2.CompressStaticCall` is `Compress` with the fast path above, written as a non-generic function: the state, the block and the result are concrete `koalabear.Element` arrays, and every call, including the one to gnark-crypto's permutation, is static. It falls back to `Compress` for any permutation other than `KoalaBearAsm`. This is the same reason gnark-crypto's `CompressPoseidon2` allocates nothing: the compiler can see every function the arrays are passed to. The KoalaBear compressor, MD hasher and transcript built on it are benchmarked as `generic-asm-static-call`

in runs 5 and 6. Baseline and `generic-asm` were re-run alongside them and moved by at most 3% from runs 3 and 4, so each `generic-asm-static-call` ratio is taken against the baseline of its own run.

It makes 0 allocations per compression (2 for `generic-asm`) and 1 per MD hash instead of $n/4 + 1$. It is within 2% of baseline on every single-threaded benchmark (Table 4). With the GC on, `CompressParallel` runs at 14.2 ns at 64 threads against 12.5 ns for baseline, compared with 103 ns for `generic-asm` (Table 5). FiatShamir allocations drop by 50–93% (from 698 to 346 at rounds=16, upd=8, and from 76 066 to 5 387 at rounds=256, upd=1024). The rest are allocated by the transcript itself, not by the compression.

8.5 Caveats

- The benchmark is close to a worst case. In the prover the live heap is much larger, so with `GOGC=100` cycles are much rarer (but each one marks more). The real impact depends on the prover's heap and its other allocations. The fix is the same either way: make `Compress` allocation-free.
- Above 32 threads the extra threads are hyperthreads sharing a core. Even baseline gains only 1.7× from 32 to 64 threads.
- The harness runs its own loop rather than the benchmark; at 64 threads it measures 106 ns with the GC on and 16 ns off, against 103 and 14.6 ns for the benchmark. Tracing and profiling add overhead, so the traced and profiled runs are a little slower again (105–143 ns). Comparisons are always between runs with identical settings.
- CPU profiles at 64 threads (`report/data/profile-64-gc{on,off}.txt`) agree with the analysis above: with the GC on, 22 CPU-seconds were sampled in 2.5 s (about 9 threads' worth), against 236 in 4.9 s with it off. About 10% of the sampled CPU is in `mallocgc` and only a few percent in marking.
- One part of each stop-the-world window is not explained by the trace (Section 8.3). This does not change the conclusion: removing the allocations removes the cycles, so the windows go away too.
- The same two allocations account for the 16 385 allocations of MDHasher at $n = 65\,536$, so they also matter single-threaded at a smaller scale (7–11%).

9 Recommendations

1. **Make `KoalaBearAsm` the default** when the CPU supports AVX-512 and the parameters are 6 full and 21 partial rounds; otherwise fall back to the scalar branch.
2. **Remove the two allocations in `Compress`** by applying the same KoalaBear branch there: reinterpret `x` and `res` as KoalaBear arrays and do the feed-forward with direct calls. A per-permutation scratch buffer would not work, as it would break concurrent use.
3. **Drop `KoalaBearVector`**, or keep it only as a recorded negative result.
4. **For MamaBear** (about 3×, with no assembly available), the overhead is purely the generic code. Closing it means concrete branches like KoalaBear's, which would also be the place to replicate gnark-crypto's `sumState` (Section 6.1).
5. Look into the higher allocation count of `current` in FiatShamir (about 2.9× baseline).

A Reproducing

From `prover-ray/`, with `benchstat` installed (`go install golang.org/x/perf/cmd/benchstat@latest`):

```
# all fields, medium scale (a few minutes)
scripts/bench-generic-poseidon.sh medium <outdir>

# KoalaBear only, as in run 3 (about 80 s)
go test ./crypto/genericbench/ -run TestAgreement
GENERIC_BENCH_SCALE=medium go test ./crypto/genericbench/ -run '^$' \
  -bench '/field=koalabear/' -benchmem -count 6 -benchtime 200ms > raw.txt
benchstat -col /impl raw.txt
```

B What it means that a variable escapes

Escape analysis is how the Go compiler decides, for each local variable, whether it can live on the *stack* or must go on the *heap*. A variable *escapes* when it is moved to the heap.

B.1 Stack vs. heap

- **Stack:** memory that belongs to one function call. It is reserved when the function starts and released when it returns. It costs almost nothing and the garbage collector never sees it.
- **Heap:** memory that can outlive the function call. Each allocation has a cost, and the garbage collector later has to find and free it.

The compiler keeps a local on the stack only if it can prove that nothing still refers to it once the function returns. If it cannot prove that, it moves the variable to the heap. With `-gcflags=-m`, it prints `moved to heap: x`.

The usual ways a variable escapes:

- returning a pointer to it;
- storing a pointer to it in a global, a struct field or a channel;
- passing a pointer to it into a function the compiler cannot see into, so it has to assume the worst.

B.2 The function where `x` escapes: `Compress`

Listing 10: Generic `Compress` (`crypto/poseidon2/hasher.go`).

```
func Compress[T any, PT generic.Element[T]](perm *Permutation[T, PT], state, block Octuplet[T]) Octuplet[T]
    {
      var x [CompressionWidth]T // local array: 16 elements
      copy(x[:BlockSize], state[:])
      copy(x[BlockSize:], block[:])
      if err := perm.Permute(x[:]); err != nil { // (1) pointer to x leaves Compress
            panic(err)
      }
      res := block // local array: 8 elements
      for i := range res {
            PT(&res[i]).Add(&res[i], &x[BlockSize+i]) // (2) pointers to res and x go to a PT method
      }
      return res
}
```

`x` and `res` are both local arrays, so ideally they would live on the stack. But pointers to them leave `Compress` in two places.

(1) `perm.Permute(x[:])`. `x[:]` is a slice: a pointer to `x`'s memory plus a length. So the compiler has to look at what `Permute` does with its `input` parameter:

Listing 11: Generic `Permute` (`crypto/poseidon2/permutation.go`, excerpt).

```
func (h *Permutation[T, PT]) Permute(input []T) error {
      ...
      if h.kbAsm != nil {
            return h.kbAsm.Permutation(asKB(input)) // branch taken at run time for generic-asm
      }
      h.matMulExternal(input) // other branches: generic helpers
      for r := range ... {
            h.addRoundKey(r, input)
            ...
            h.sBox(&input[0]) // e.g. sBox -> PT(x).Mul(x, &sq)
            ...
      }
}
```

The non-asm branches pass `input`, or pointers into it, to helpers that call `PT(...)` methods like `Mul` and `Add`. Because `Compress` and `Permute` are generic, Go compiles one shared version of each for every element type with the same memory layout. In that shared version, `PT` methods are called through a lookup table (the generic dictionary). The compiler does not know which concrete method will run, so it assumes the method might keep the pointer. It therefore marks `input` as escaping, and since `x[:]` was passed as `input`, `x` is moved to the heap.

At run time `generic-asm` always takes the `kbAsm` branch, which does not keep the pointer. That does not help, because the decision is made once at compile time and covers every branch of `Permute`.

(2) `PT(&res[i]).Add(&res[i], &x[BlockSize+i])`. This is the same problem without a nested call: pointers to `res` (and to `x`) go straight to a method called through the dictionary. So `res` is moved to the heap too.

The compiler confirms both (`-gcflags=-m`):

```
crypto/poseidon2/hasher.go:26:6: moved to heap: poseidon2.x
```

```
crypto/poseidon2/hasher.go:32:2: moved to heap: poseidon2.res
```

That is the 2 allocations (96 B) per compression for `generic`, `generic-vec` and `generic-asm`.

B.3 The same work without escaping: `kbCompressAsm`

Listing 12: `kbCompressAsm` (`crypto/poseidon2/koalabear.go`), used by `generic-asm-static-call`.

```
func kbCompressAsm(perm *kbposeidon2.Permutation, state, block []koalabear.Element) {
	var x [CompressionWidth]koalabear.Element // concrete type
	copy(x[:BlockSize], state)
	copy(x[BlockSize:], block)
	if err := perm.Permutation(x[:]); err != nil { // concrete gnark-crypto method
		panic(err)
	}
	for i := range BlockSize {
		state[i].Add(&block[i], &x[BlockSize+i]) // concrete koalabear.Element.Add
	}
}
```

It does the same steps, but nothing here is generic:

- **`perm.Permutation(x[:])`** is a direct call to a known gnark-crypto method. The compiler analyses it and finds it does not keep `input`.
- **`state[i].Add(...)`** is a direct call to `koalabear.Element.Add`, which does not keep its arguments either.
- **No separate `res` array.** The result goes into `state`, and `CompressStaticCall` returns that by value.

The compiler can follow every pointer, so it proves `x` dies when the function returns and keeps it on the stack. The result is 0 allocations, the same as gnark-crypto's own `CompressPoseidon2`.

B.4 Why it matters for the benchmark

- **Single-threaded:** the 2 heap allocations per call cost a few percent.
- **In `CompressParallel`:** 64 goroutines allocate constantly. The garbage collector runs all the time, and they compete for it. That is why `generic-asm` ends up at 103 ns per compression, against 12.5 ns for baseline and 14.2 ns for `generic-asm-static-call` (Table 5).

C Function closures

A closure is a function value that captures variables from the place it was defined. It can still use those variables later, after the function that created it has returned.

Listing 13: `StaticCallCompressor` (`crypto/poseidon2/hasher.go`).

```
func StaticCallCompressor[T any, PT generic.Element[T]](perm *Permutation[T, PT]) CompressFunc[T] {
	return func(state, block Octuplet[T]) Octuplet[T] {
		return CompressStaticCall(perm, state, block) // uses perm from the outer function
	}
}
```

- The inner `func(state, block ...)` `...` is a function literal, a function without a name.
- It uses `perm`, which is not one of its own parameters: `perm` belongs to `StaticCallCompressor`.
- `StaticCallCompressor` returns this function. Later, when the benchmark calls `c.compress(st, blk)`, `StaticCallCompressor` has long since returned, but the function still has its `perm`.

C.1 How Go implements it

A closure value is a pointer to a small object that holds a pointer to the function's compiled code and the captured variables, here `perm`. Roughly:

```
type closure struct {
	fn *code // CompressStaticCall wrapper
	perm *Permutation[T, PT] // captured
}
```

That object has to outlive `StaticCallCompressor`, so it goes on the heap; the compiler reports `func literal escapes to heap` (Appendix B). It is allocated once, when `StaticCallCompressor` is called. Every later call reuses it, which is why it does not appear in the per-call allocation counts.

C.2 Why the code uses one

`CompressFunc[T]` is defined as `func(state, block Octuplet[T]) Octuplet[T]`, which has no room for a `perm` argument. The closure fixes `perm` in advance, so the result fits that signature.

- **What it allows:** the hasher, the transcript and the benchmark can all take any `CompressFunc`. That includes gnark-crypto's `kbvortex.CompressPoseidon2`, a plain function with no captured variables, or one of these closures. That is how the benchmark puts baseline and generic implementations in the same `compressors` list.
- **The cost:** each call goes through a function pointer, so the compiler cannot inline it. That is a nanosecond or so, small next to a Poseidon2 permutation.